

Lab 2.5 - Roofline Analysis of A64FX

Advanced Computer Architectures

University of Minho

Lab 2.5 focuses on the manual creation of a Roofline model of a server for a sample application. Use a cluster node (through `sbatch` script submission) to run the analysis using the command line.

The goal of Lab 2.5 is to evaluate the maximum computing throughput and memory bandwidth of a computing node, while identifying the relevant ceilings that may impact performance of a sample application. The performance data gathered from an application allows it to be plotted into a Roofline graph for a given node, in order to assess its optimisation efficiency (how far it is from a theoretical maximum). The relative position of an application arithmetic intensity relatively to the computing node roof and ceilings gives an insight into the inefficiencies of the code, whose limitations can be better identified and optimised.

To load the compiler in the environment use these commands (use these or newer versions of each software):

GNU Compiler (for several system libraries): `module load GCC/13.3.0`

Consider the code available in the `/projects/F202500010HPCVLABUMINHO/ACA/lab2.5` folder in the Deucalion supercomputer.

1. Benchmarking Memory Throughput with STREAM

Lab 1.1 Copy STREAM's source code to your home. It is available at github:

<https://github.com/jlinford/stream>

Compile the code on an interactive session in an ARM compute node at Deucalion. Use STREAM to assess the bandwidth of L1 and L2 caches, and HBM as well. The analysis should be the same to evaluate each level of the memory hierarchy, operating on datasets appropriate to each. Note that the highest bandwidth is only achieved using either `stream_openmp`, `stream_zfill` or `stream_zfill_acle`. Save the results to be used on exercise 1.3.

2. Benchmarking Compute Throughput with HPL

Lab 1.2 Assess the maximum computational throughput, as GFlops/s, of the A64FX compute nodes using High Performance Linpack (HPL). Do this analysis for a single node and measure the results, which will be used on exercise 1.3.

You need to use Fujitsu's compiler: `module load FJSVstclanga`

Access and ARM compute node through an interactive SLURM session:

```
srun --partition=dev-arm --account=f202500010hpcvlabuminhoa --nodes=1 --ntasks=1
--time=00:30:00 --pty bash
```

To configure and install HPL you must follow these steps:

```
./configure CC=mpifcc --prefix=/home/USER/lab2.5/hpl-arm/ CFLAGS="-O3 -ffast-math
-Kopenmp -SSL2 -SSL2BLAMP -march=native -funroll-loops"
make -j 44 && make install
```

Finally, execute the benchmark by submitting a **copy** of the batch script available at: `/projects/F202500010HPCVLABUMINHO/ACA/lab2.5/job-hpl.sh` **Modify the EXE accordingly!!!!**

Does the performance measured match the advertised theoretical maximum?

3. Plotting the Roofline

1.3 Consider the memory bandwidth and computational throughput measured in exercises 1.1 and 1.2. Use those values to plot the Roofline for the A64FX chip using the python script (on your laptop), which can be obtained by:

```
scp -r <login-deucalion>:/projects/F202500010HPCVLABUMINHO/ACA/lab2.5/
roofline-plotter .
```

Identify and calculate the impact of the relevant memory and compute ceilings that may impact the performance of the parallel matrix multiplication code developed in the first lab session. Add those ceilings to the Roofline (see the python script provided).

On Deucalion, measure the performance and arithmetic intensity of your parallel matrix multiplication on the A64FX chip for 3 dataset sizes (fitting L1, L2, and HBM) for 1 and 48 threads. Note that the GFlops/s can be measured through performance counters (remember `papi_avail` and `perf?`) Plot those measurements on the Roofline and identify the factors limiting the performance of each version of the code.